CSC 241/ COG 241
ABSTRACT DATA TYPES AND PROGRAMMING METHODOLOGY

Lecture09

Recursive Programming

AGENDA

- Intro to Recursion
- Recursion Examples

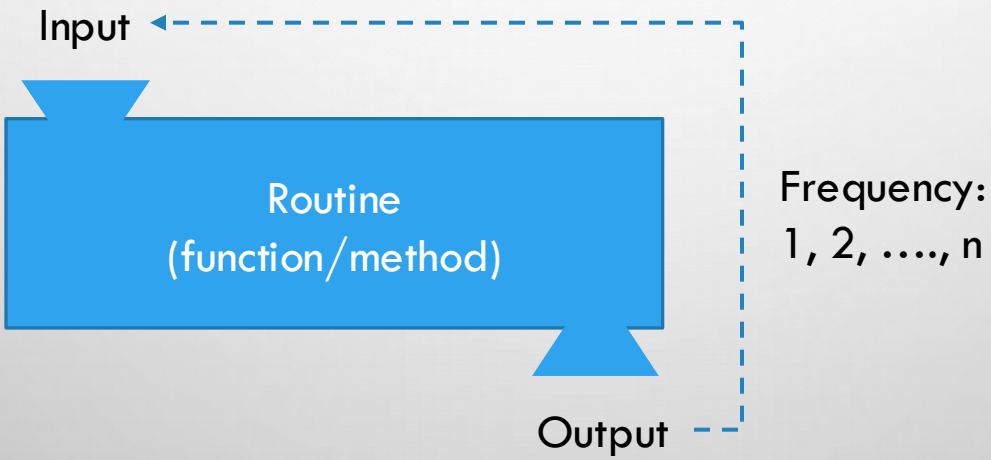
REFERENCES

- Introduction to Computer Science using Java (reference)
  - Chapter 90~91
- Oracle Java Tutorials (reference)

INTRO TO RECURSION

REPETITION OF ROUTINE

- Iteration
  - By a *loop* statement, a particular routine is repeated
  - Frequency matters
  - Given data does not change



EXAMPLE WITH ITERATION

- Consider a function to get a sum of n non-negative integers
  - $sum = 0 + 1 + 2 + \cdots + n$
- Example code:

```
public int sum (int number) {
    int result = 0;
    for (int i = 0; i <= number; i++) {
        result += i;
    }
    return result;
}

public static void main (String args[]) {
    Scanner input = new Scanner(System.in);
    int nInput = input.nextInt();
    System.out.println(”Sum(” + nInput + “)“ + sum(nInput));
}
```

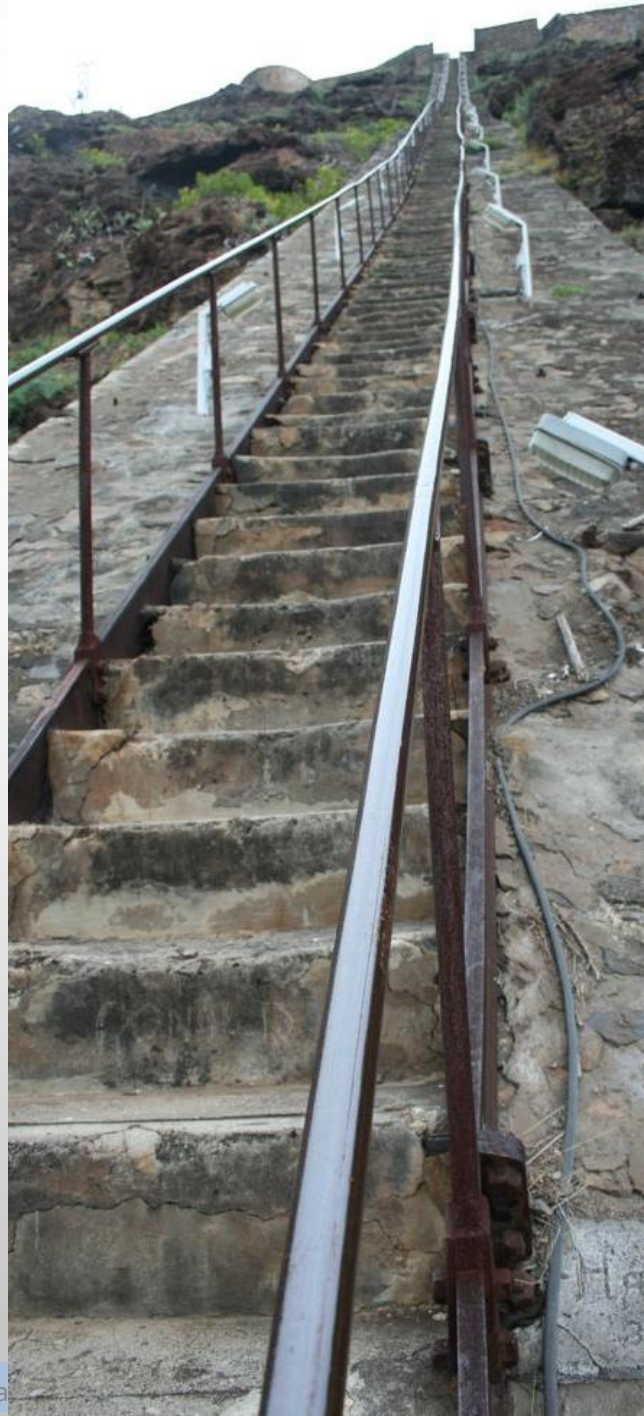
ALTERNATIVE PERSPECTIVE FOR REPETITION

"A journey of a 1000 feet begins with a single step"

Lao-Tse(Lao-tzu)

- Two perceptions
  - 1000 feet is a gathering of 1000 single steps

    → Repetition of one single step
  - A singlet step may finish the journey



OBSERVATION BASED ON ITERATION

- $sum = 0 + 1 + 2 + \cdots + n$
  - A single number is added in each loop

index	number	sum
0	0	0
1	1	1
2	2	3
3	3	6
4	4	10
…	…	…
$n-1$	$n-1$	$\frac{n(n-1)}{2}$
n	n	$\frac{n(n+1)}{2}$

NEW OBSERVATION

- $sum(n) = \underbrace{0 + 1 + 2 + \cdots + (n-1)} + n$

$$sum(n-1) \quad + n$$

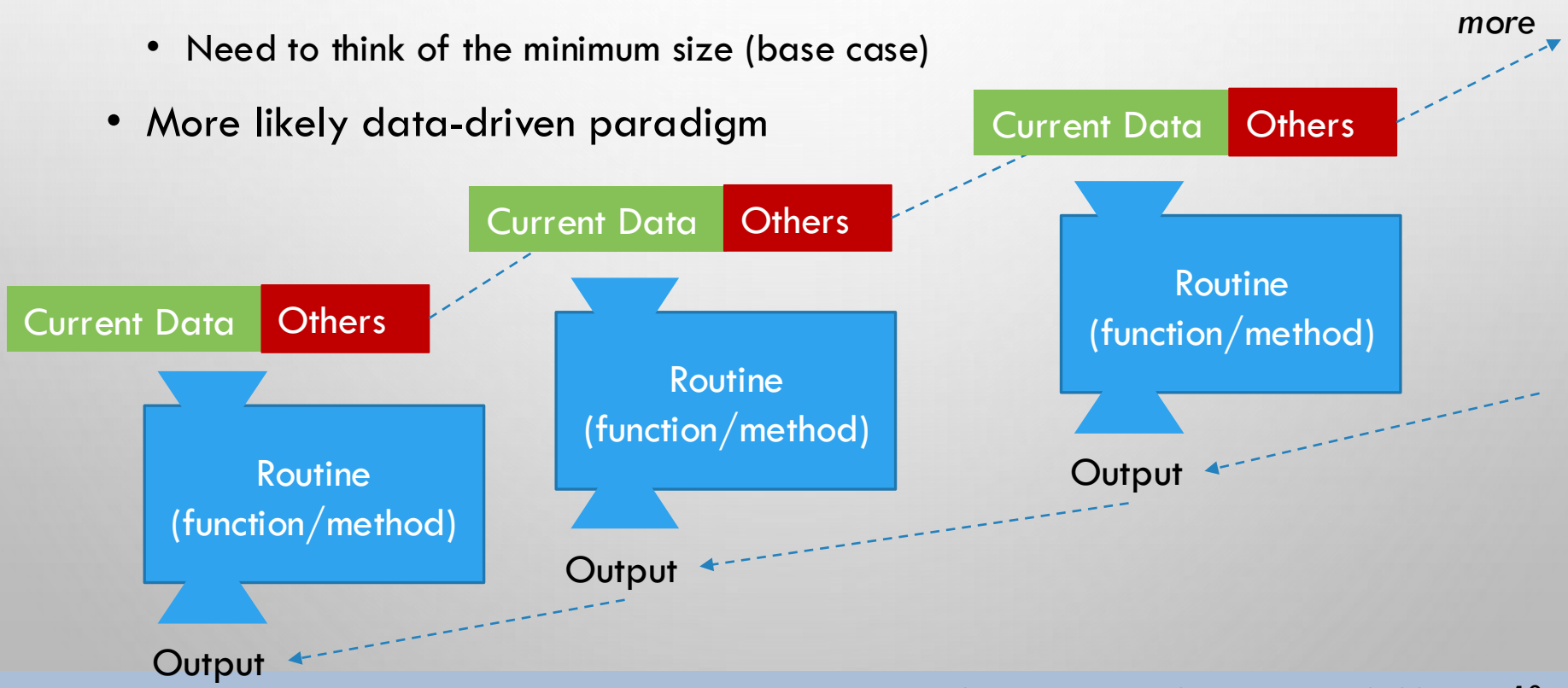
$$\underbrace{sum(n-2)} \quad + (n-1) + n$$

$$\ldots$$

- We run the method `sum` with different data $(n, n-1, n-2, \ldots, 0)$
  - We call this strategy recursion

REPETITION BY RECURSION

- Recursion
  - No loop
  - Call the same routine (function/method) recursively
    - In each call, size of data should change (usually smaller)
    - Need to think of the minimum size (base case)
  - More likely data-driven paradigm



STRUCTURE FOR RECURSION

- Base case
  - It has minimum data
  - It represents conditions for stopping dividing data
  - There would be multiple base cases
- Recursive call
  - It recursively calls the routine with a different data

EXAMPLE FOR RECURSION

- Have a recursive method for sum of n integers.
  - $sum = 0 + 1 + 2 + \cdots + n$
- Example code:

```
public int sum (int number) {
    if (number == 0)     return 0; // base case
    else
        return number + sum(number - 1);
}

public static void main (String args[]) {
    Scanner input = new Scanner(System.in);
    int nInput = input.nextInt();
    System.out.println(”Sum(” + nInput + “)“ + sum(nInput));
}
```

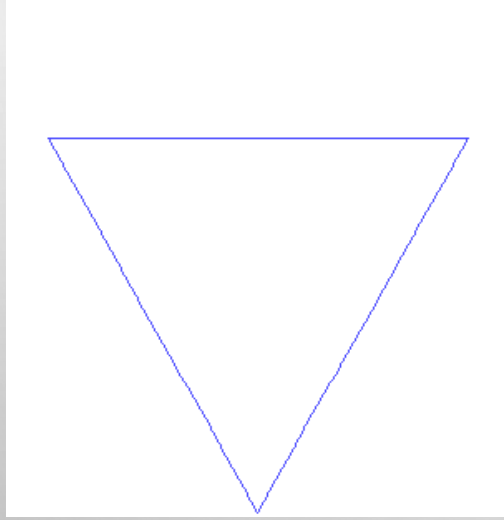
WHY RECURSION?

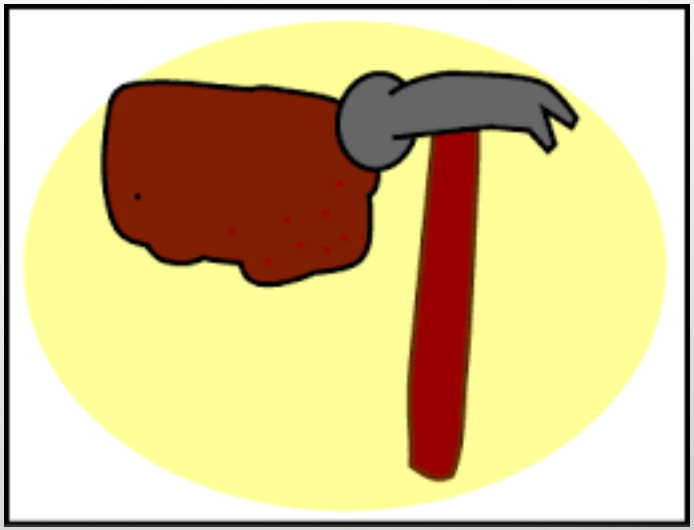
- It divides a big data to multiple smaller data
  - A smaller data is easy to solve
  - It takes quick to solve a smaller data
- It may simplify programming
  - Shorter code
- But there would be potential issues
  - Overhead for recursive-calling the same routine
  - Higher spatial complexity
  - So, programing with iteration may be needed or advantageous in a particular domain or problem

RECURSION EXAMPLES

EXAMPLES IN THE REAL WORLD

- Popular and common examples
  - Drawing a snowflake
  - Pounding a rock to dust with a hammer
    - (Reference)





MORE EXAMPLES IN PROGRAMMING

- Recursive Binary Search
- Recursive Insertion Sort
- Recursive Merge Sort
- Try to implement it by yourself!

ITERATIVE BINARY SEARCH

```
int binarySearch(int [] arr, int key) {
    int first = 0;
    int last = arr.length-1;
    int mid;
    while (first < last) {
        mid = (first + last)/2;
        if (key > arr[mid])
            first = mid + 1;
        else
            last = mid;
    }
    if (key = arr[first])
        return first;
    else
        return -1;
}
```

RECURSIVE BINARY SEARCH

```
int binarySearch(int[] arr, int item, int first, int last){
    int mid;
    if(first > last)                // base case 1
        return -1;
    else {
        mid = (first + last)/2;
        if(item < arr[midPoint])
            return binarySearch(arr, item, first, midPoint-1);
        else if (item == arr[mid]) {  // base case 2
            item = arr[mid];
            return mid;
        }
        else
            return binarySearch(arr, item, mid+1, last);
    }
}
```

ITERATIVE INSERTION SORT

```
void insertionSort(int [] arr) {
    int size = arr.length;
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            --j;
        }
        arr[j+1] = key;
    }
}
```

- Write it in a recursive method:
  - (Reference)

MERGE SORT

PROCEDURE *Merge-Sort*

INPUT *array* A: $a_1, a_2, \ldots, a_n$; *index* p, r

if $p < r$

$q \leftarrow \lfloor {}^{(p+r)}/_{2} \rfloor$

Merge-Sort(A, p, q)

Merge-Sort$(A, q+1, r)$

Merge(A, p, q, r)

end for

- Java code: (Reference)

MERGE

- Linearly merging two arrays

PROCEDURE *Merge*

INPUT *array* A: $a_1, a_2, \ldots, a_n$; *index* p, q, r

$n_1 \leftarrow q - p + 1,\ n_2 \leftarrow r - q$

Create $L[1 \text{ to } n_1] \leftarrow A[1 \text{ to } q]$, $L[n_1 + 1] \leftarrow \infty$

Create $R[1 \text{ to } n_2] \leftarrow A[q + 1 \text{ to } r]$, $R[n_2 + 1] \leftarrow \infty$

$i \leftarrow 1, j \leftarrow 1$

for $k \leftarrow p$ **to** r

    if $L[i] \leq R[j]$

        $A[k] \leftarrow L[i]$

        $i \leftarrow i + 1$

    else

        $j \leftarrow j + 1$

    end if

end for

PROGRAMMING EXERCISE

- Change `MyClass.java`
- First, define a JSON format for student's data: `myclass.json`
- Write recursive search/sort methods